

Le basi

Sintassi di base

Le regole fondamentali della sintassi Python — come "parla" il linguaggio.

Perché le regole esistono?

Ogni linguaggio, anche quelli umani come l'italiano, ha delle regole grammaticali. Se scrivi "io mangio la pizza" funziona, ma se scrivi "pizza la mangio io" il significato cambia, e in certi casi non si capisce più nulla.

Python funziona allo stesso modo: ha delle **regole di sintassi** che devi rispettare. Se le rompi, Python si ferma e ti mostra un errore. Non perché sia cattivo, ma perché non riesce a capire cosa vuoi fare.

L'indentazione: gli spazi contano!

Questa è la regola più importante e più diversa rispetto ad altri linguaggi.

In Python, gli **spazi all'inizio di una riga** non sono opzionali o decorativi: **fanno parte del codice**. Servono a dire a Python quali istruzioni stanno "dentro" ad un blocco (come un `if` o un ciclo) e quali invece stanno "fuori".

Immagina di scrivere una ricetta:

```
Se hai la farina:  
    mescola con l'acqua  
    aggiungi il sale  
prepara il forno
```

Le righe rientrate (con gli spazi davanti) appartengono alla condizione "se hai la farina". L'ultima riga invece vale sempre, indipendentemente dalla farina. Python funziona esattamente così.

```
if 5 > 3:  
    print("Cinque è maggiore di tre") # questa riga è DENTRO l'if  
print("Questa riga è FUORI dall'if")  # questa viene eseguita sempre
```

Lo standard è usare **4 spazi** per ogni livello di rientro. Non mescolare spazi e tabulazioni (il tasto Tab): causa problemi.

Se dimentichi il rientro, Python ti avvisa con un errore chiarissimo:

```
if 5 > 3:
print("Errore!") # IndentationError: expected an indented block
```

Maiuscole e minuscole

Python distingue sempre tra lettere maiuscole e minuscole. Questo si chiama essere **case-sensitive**.

Quindi `nome`, `Nome` e `NOME` sono **tre cose completamente diverse** per Python:

```
nome = "Alice"
Nome = "Bob"
print(nome) # Alice
print(Nome) # Bob
```

Fai attenzione anche a `True` e `False`: devono avere la prima lettera maiuscola. `true` o `TRUE` causano un errore.

Istruzioni su più righe

Di solito scrivi un'istruzione per riga. Se un'istruzione è molto lunga e vuoi spezzarla, puoi usare il carattere `\` alla fine della riga per dire a Python "continua alla riga successiva":

```
risultato = 10 + 20 + \
            30 + 40
print(risultato) # 100
```

Oppure, se apri una parentesi, Python capisce da solo che l'istruzione non è ancora finita:

```
numeri = (1, 2, 3,
          4, 5, 6)
```

Il punto e virgola (non è necessario)

In alcuni linguaggi, ogni istruzione deve finire con `;`. In Python **non serve**. Una riga = un'istruzione, e basta.

Potresti scrivere più istruzioni sulla stessa riga usando `;`, ma è una cattiva abitudine che rende il codice difficile da leggere:

```
# Funziona, ma è meglio evitarlo
a = 1; b = 2; print(a + b)

# Molto meglio così:
a = 1
b = 2
print(a + b)
```

Le parole riservate (keywords)

Python ha alcune parole speciali che hanno già un significato preciso nel linguaggio. Non puoi usarle come nomi di variabili.

Per esempio, `if` significa "se" per Python. Se provi a creare una variabile chiamata `if`, Python si confonde e dà errore.

Queste parole si chiamano **keyword**:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Non devi memorizzarle tutte ora. Le imparerai una alla volta quando ne avrai bisogno. L'importante è sapere che esistono: se Python ti dà un errore strano su un nome di variabile, controlla che non sia una keyword.

Booleani

I valori vero/falso in Python — la base di ogni decisione nel codice.

Cos'è un booleano?

Nella vita di tutti i giorni facciamo continuamente valutazioni del tipo "vero o falso":

- La luce è accesa? Sì o No.
- Sono maggiorenne? Vero o Falso.
- Fa freddo fuori? Vero o Falso.

In Python esistono esattamente questi due valori: **True** (vero) e **False** (falso). Il tipo di dato che li contiene si chiama **booleano** (`bool`).

Attenzione: la prima lettera è **sempre maiuscola**. `true` e `false` (tutto minuscolo) causano un errore.

```
luce_accesa = True
fa_freddo = False
print(type(luce_accesa))  # <class 'bool'>
```

I confronti producono booleani

Ogni volta che confronti due valori, Python restituisce **True** o **False** :

```
print(5 > 3)      # True  - 5 è maggiore di 3? Sì
print(5 < 3)      # False - 5 è minore di 3? No
print(5 == 5)     # True  - 5 è uguale a 5? Sì
print(5 != 3)     # True  - 5 è diverso da 3? Sì
print(5 >= 5)     # True  - 5 è maggiore o uguale a 5? Sì
print(5 <= 4)     # False - 5 è minore o uguale a 4? No
```

⚠ `=` e `==` sono cose diverse!

- `=` assegna un valore a una variabile: `x = 5`
- `==` confronta due valori: `x == 5` (è uguale a 5?) ⚠

Combinare condizioni con **and**, **or**, **not**

Spesso vuoi combinare più condizioni insieme. In Python si usano tre operatori logici.

and — "e"

Restituisce `True` solo se **tutte** le condizioni sono vere. Come il semaforo verde: puoi passare solo se la strada è libera E il semaforo è verde.

```
eta = 20
ha_patente = True

# Puoi guidare solo se sei maggiorenne E hai la patente
print(eta >= 18 and ha_patente) # True
```

or — "o"

Restituisce `True` se **almeno una** condizione è vera. Come entrare in un cinema: puoi entrare se hai il biglietto O sei un membro del club.

```
ha_biglietto = False
e_membro = True

print(ha_biglietto or e_membro) # True — basta una delle due
```

not — "non"

Inverte il valore: trasforma `True` in `False` e viceversa.

```
piove = False
print(not piove) # True — NON piove, quindi posso uscire

maggiorenne = True
print(not maggiorenne) # False
```

Valori "truthy" e "falsy"

In Python, non solo `True` e `False` possono essere trattati come booleani. **Ogni valore** può essere interpretato come vero o falso.

I valori che Python considera **falsi** (equivalenti a `False`):

- `False`
- `0` e `0.0`
- `""` (stringa vuota, cioè nessun testo)
- `[]` (lista vuota)
- `None` (assenza di valore)

Tutto il resto è considerato **vero**. Un numero diverso da zero, una stringa con del testo, una lista con elementi.

```
print(bool(0))      # False – zero è falso
print(bool(""))     # False – stringa vuota è falsa
print(bool([]))     # False – lista vuota è falsa

print(bool(42))     # True  – qualsiasi numero non-zero è vero
print(bool("ciao")) # True  – qualsiasi testo non-vuoto è vero
print(bool([1, 2])) # True  – una lista con elementi è vera
```

Questo torna molto utile nelle condizioni. Per esempio, puoi controllare se un nome è stato inserito così:

```
nome = ""
if nome:
    print("Ciao, " + nome + "!")
else:
    print("Non hai inserito nessun nome.") # questa viene stampata
```

Commenti in Python

Come lasciare note nel codice per te stesso e per chi leggerà il tuo lavoro.

Perché scrivere commenti?

Immagina di aprire un tuo vecchio quaderno di appunti dopo sei mesi. Se hai scritto solo formule e numeri senza spiegazioni, probabilmente non ricordi più cosa significano.

Nel codice succede la stessa cosa. Un **commento** è una nota che scrivi nel codice per spiegare cosa sta succedendo. Python ignora completamente i commenti quando esegue il programma: sono messaggi solo per gli esseri umani.

Scrivere buoni commenti è una delle abitudini più importanti di un programmatore.

Commenti su una riga

In Python, un commento inizia con il simbolo `#`. Tutto quello che viene scritto dopo `#` sulla stessa riga viene ignorato da Python:

```
# Questo è un commento: Python lo ignora completamente
print("Ciao!") # Puoi mettere un commento anche dopo il codice

# Puoi usare i commenti per "disattivare" del codice temporaneamente:
# print("Questa riga non viene eseguita")
```

Commenti su più righe

Se vuoi scrivere un commento lungo, hai due opzioni.

Opzione 1: Metti `#` all'inizio di ogni riga

```
# Questo programma calcola l'area di un rettangolo.
# Prende la base e l'altezza come input
# e stampa il risultato.
```

Opzione 2: Usa le virgolette triple `"""..."""`

```
"""
```

```
Questo è un blocco di testo su più righe.
```

```
Python non lo esegue come codice.
```

```
Viene spesso usato all'inizio di un file per descriverlo.
```

```
"""
```

Tecnicamente le virgolette triple creano una stringa di testo, non un vero commento. Ma se non la assegna a nessuna variabile, Python la ignora lo stesso. Vengono usate moltissimo in pratica.

Cosa scrivere (e cosa NON scrivere)

Un buon commento spiega il **perché** di una scelta, non il **cosa** fa il codice (quello si dovrebbe capire già leggendo il codice).

```
# Male: il commento dice l'ovvio, non aggiunge nulla
```

```
x = 10 # assegna 10 a x
```

```
# Bene: spiega perché usiamo quel numero specifico
```

```
VELOCITA_MASSIMA = 130 # limite autostradale in Italia, in km/h
```

```
# Bene: spiega una formula non ovvia
```

```
area = 3.14159 * raggio ** 2 # formula del cerchio: pi * r2
```

Le docstring: commenti per le funzioni

Le **docstring** sono commenti speciali che si mettono subito dentro una funzione (o una classe) per spiegare cosa fa. Si scrivono con le virgolette triple:


```
def calcola_area(base, altezza):  
    """Calcola e restituisce l'area di un rettangolo."""  
    return base * altezza  
  
def saluta(nome):  
    """  
    Stampa un messaggio di benvenuto personalizzato.  
  
    Parametri:  
    - nome: il nome della persona da salutare  
    """  
    print("Ciao, " + nome + "!")
```

Le docstring sono utili anche perché puoi leggerle dal programma stesso con la funzione `help()` :

```
help(calcola_area)  
# Stampa: Calcola e restituisce l'area di un rettangolo.
```

Tipi di dati in Python

I diversi tipi di informazione che Python sa gestire.

Perché esistono i tipi di dati?

Il computer gestisce informazioni molto diverse tra loro: numeri, testi, valori vero/falso. Non ha senso sommare del testo come se fossero numeri, o confrontare un nome con un prezzo.

Per questo ogni valore in Python ha un **tipo di dato**: una categoria che dice a Python com'è fatto quel valore e cosa puoi farci.

I tipi principali

Numero intero — `int`

Un numero senza virgola. Può essere positivo, negativo o zero.

```
eta = 16
temperatura = -5
anno = 2024
milione = 1_000_000 # puoi usare _ come separatore per leggere meglio i
grandi numeri

print(type(eta)) # <class 'int'>
```

Numero decimale — `float`

Un numero con la virgola. In Python (e nella programmazione in generale) si usa il **punto** invece della virgola.

```
altezza = 1.75
pi_greco = 3.14159
temperatura = -2.5

print(type(altezza)) # <class 'float'>
```

Testo — `str` (stringa)

Qualsiasi sequenza di caratteri: lettere, numeri, simboli. Si scrive tra virgolette singole `'...'` o doppie `"..."` :

```
nome = "Alice"
messaggio = 'Ciao, mondo!'
frase = """Questa è una stringa
su più righe."""

print(type(nome)) # <class 'str'>
```

Il termine "stringa" deriva dall'inglese *string* (filo), perché è come una sequenza di caratteri infilati uno dopo l'altro.

Vero o Falso — `bool` (booleano)

Solo due valori possibili: `True` (vero) o `False` (falso). Nota la lettera maiuscola.

```
maggiorenne = True
piove = False

print(type(maggiorenne)) # <class 'bool'>
```

Assenza di valore — `None`

`None` è un valore speciale che significa "niente". È utile per indicare che una variabile esiste ma non contiene ancora nessun dato.

```
risultato = None

print(type(risultato)) # <class 'NoneType'>
```

È come una scatola vuota: la scatola c'è, ma non c'è niente dentro.

Tipi per raccogliere più valori insieme

Oltre ai tipi semplici, Python ha tipi che contengono più valori contemporaneamente. Li vedremo in dettaglio nei capitoli successivi, ma è bene sapere che esistono:

Tipo	Esempio	Cosa è
<code>list</code> (lista)	<code>[1, 2, 3]</code>	Una sequenza ordinata di elementi, modificabile
<code>tuple</code>	<code>(1, 2, 3)</code>	Come una lista, ma non modificabile
<code>set</code> (insieme)	<code>{1, 2, 3}</code>	Una raccolta senza duplicati e senza ordine fisso
<code>dict</code> (dizionario)	<code>{"nome": "Alice"}</code>	Coppie chiave-valore, come un vocabolario

Come Python assegna il tipo automaticamente

In Python non devi dichiarare esplicitamente il tipo di una variabile: Python lo intuisce dal valore che assegna.

```
x = 10      # Python capisce che è un int
y = 3.14    # Python capisce che è un float
z = "ciao"  # Python capisce che è una str
```

Questo si chiama **tipizzazione dinamica**: il tipo non è fisso, e puoi anche cambiarlo nel corso del programma (anche se di solito non è una buona idea).

Riepilogo rapido

intero	= 10	# int	– numero senza virgola
decimale	= 3.14	# float	– numero con virgola
testo	= "Python"	# str	– testo
vero_o_falso	= True	# bool	– vero o falso
niente	= None	# None	– assenza di valore
lista	= [1, 2, 3]	# list	– sequenza modificabile
tupla	= (1, 2, 3)	# tuple	– sequenza fissa
insieme	= {1, 2, 3}	# set	– raccolta senza duplicati
dizionario	= {"a": 1}	# dict	– coppie chiave-valore

Operatori in Python

I simboli con cui fare calcoli, confronti e operazioni logiche in Python.

Cosa sono gli operatori?

Un **operatore** è un simbolo che esegue un'operazione su uno o più valori. Li usi già ogni giorno: `+`, `-`, `*` sono operatori matematici. In Python ce ne sono molti altri.

Operatori aritmetici — per i calcoli

Funzionano come la matematica che conosci già, con qualche aggiunta:

Operatore	Significato	Esempio	Risultato
<code>+</code>	Addizione	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione (con decimali)	<code>5 / 2</code>	<code>2.5</code>
<code>//</code>	Divisione intera (senza decimali)	<code>5 // 2</code>	<code>2</code>
<code>%</code>	Resto della divisione	<code>5 % 2</code>	<code>1</code>
<code>**</code>	Potenza	<code>5 ** 2</code>	<code>25</code>

L'operatore `//` (doppio slash) divide e butta via i decimali. È come chiedere "quante volte 2 entra in 5?" — la risposta è 2, con 1 di resto.

L'operatore `%` (modulo) restituisce solo il resto. Torna utilissimo per sapere se un numero è pari o dispari: se `numero % 2` vale `0`, il numero è pari.

```
a = 10
b = 3
print(a + b) # 13
print(a - b) # 7
print(a * b) # 30
print(a / b) # 3.3333...
print(a // b) # 3 (10 diviso 3 fa 3 con il resto)
print(a % b) # 1 (il resto è 1)
print(a ** b) # 1000 (10 alla potenza 3)
```

Operatori di confronto — per fare domande

Confrontano due valori e restituiscono sempre `True` o `False` :

Operatore	Significato	Esempio
<code>==</code>	È uguale a?	<code>5 == 5</code> → True
<code>!=</code>	È diverso da?	<code>5 != 3</code> → True
<code>></code>	È maggiore di?	<code>5 > 4</code> → True
<code><</code>	È minore di?	<code>5 < 3</code> → False
<code>>=</code>	È maggiore o uguale?	<code>5 >= 5</code> → True
<code><=</code>	È minore o uguale?	<code>5 <= 4</code> → False

```
x = 5
print(x == 5) # True
print(x != 3) # True
print(x > 4)  # True
print(x < 3)  # False
```

Operatori logici — per combinare condizioni

Servono a mettere insieme più condizioni:

Operatore	Significato
<code>and</code>	Vero solo se tutte le condizioni sono vere
<code>or</code>	Vero se almeno una condizione è vera
<code>not</code>	Inverte il valore (vero diventa falso e viceversa)

```
print(5 > 3 and 10 > 7) # True  - entrambe vere
print(5 > 3 or 10 < 7)  # True  - almeno una vera
print(not 5 > 3)        # False - 5 > 3 è True, not lo inverte
```

Operatori di assegnazione — scorciatoie per aggiornare variabili

Invece di scrivere `x = x + 3`, puoi usare la versione abbreviata `x += 3`. Significano la stessa cosa:

Abbreviazione	Equivalente completo
<code>x += 3</code>	<code>x = x + 3</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 3</code>	<code>x = x / 3</code>
<code>x //= 3</code>	<code>x = x // 3</code>
<code>x %= 3</code>	<code>x = x % 3</code>
<code>x **= 3</code>	<code>x = x ** 3</code>

```
punteggio = 100
punteggio += 50  # il giocatore ha guadagnato 50 punti
print(punteggio) # 150

punteggio -= 20  # penalità di 20 punti
print(punteggio) # 130
```

Operatori `in` e `not in` — per cercare elementi

Verificano se un valore è presente (o assente) in una lista, in una stringa o in qualsiasi altra sequenza:

```
frutta = ["mela", "banana", "ciliegia"]
print("mela" in frutta)      # True  - c'è la mela
print("uva" in frutta)      # False - non c'è l'uva
print("uva" not in frutta)   # True  - l'uva non c'è

testo = "Ciao, mondo!"
print("mondo" in testo)     # True
print("pizza" in testo)     # False
```

L'ordine delle operazioni

Come in matematica, Python esegue le operazioni in un ordine preciso. Le parentesi hanno sempre la precedenza massima:

```
print(2 + 3 * 4)    # 14 - prima la moltiplicazione, poi l'addizione
print((2 + 3) * 4)  # 20 - prima le parentesi
```

Se non sei sicuro dell'ordine, usa le parentesi: rendono il codice più chiaro e sicuro.

Conversione dei tipi

Come trasformare un valore da un tipo all'altro in Python.

Perché convertire i tipi?

Immagina di voler stampare questo messaggio: `"Hai 16 anni"`. Il numero `16` è un intero, ma per metterlo in mezzo al testo devi trasformarlo in una stringa di testo. Senza questa trasformazione, Python si ferma con un errore.

Questa operazione — trasformare un valore da un tipo all'altro — si chiama **conversione di tipo**.

Conversione automatica (implicita)

A volte Python fa la conversione da solo, senza che tu debba fare niente. Succede quando sommi un numero intero con uno decimale: Python trasforma automaticamente l'intero in decimale per poter fare il calcolo.


```
intero = 5
decimale = 2.5
risultato = intero + decimale

print(risultato)          # 7.5
print(type(risultato))    # <class 'float'>
```

Python fa questo solo quando è sicuro di non perdere informazioni.

Conversione manuale (esplicita)

Altre volte devi essere tu a dire a Python di convertire. Si usano funzioni apposite.

int() — trasforma in numero intero

Ecco come `int()` converte valori di tipo diverso in un numero intero:

```
x = int(3.9)      # 3 - la parte decimale viene TAGLIATA (non arrotondata!)
y = int("42")     # 42 - una stringa numerica diventa un numero
z = int(True)     # 1
w = int(False)    # 0

print(x, y, z, w) # 3 42 1 0
```

⚠️ `int(3.9)` restituisce `3`, non `4`. Il decimale viene tagliato, non arrotondato. Se vuoi arrotondare, usa `round(3.9)` che fa `4`.

float() — trasforma in numero decimale

Ecco come `float()` trasforma un valore in un numero con la virgola:

```
a = float(5)      # 5.0
b = float("3.14") # 3.14
d = float(True)   # 1.0

print(a, b, d)    # 5.0 3.14 1.0
```

str() — trasforma in testo

Questa è la conversione più usata, perché spesso devi mettere numeri dentro messaggi di testo:

```
eta = 16
# print("Ho " + eta + " anni")      # ERRORE — non puoi sommare str e int
print("Ho " + str(eta) + " anni")    # OK — "Ho 16 anni"

n = str(42)      # "42"
f = str(3.14)    # "3.14"
b = str(True)    # "True"
```

In alternativa alle conversioni manuali, puoi usare le **f-string** (stringhe formattate), che gestiscono tutto automaticamente:

```
eta = 16
print(f"Ho {eta} anni") # "Ho 16 anni" — più comodo!
```

bool() — trasforma in vero/falso

Ogni valore in Python può essere convertito in booleano. I valori "vuoti" o "zero" diventano **False**, tutto il resto diventa **True** :

```
print(bool(0))      # False – zero è falso
print(bool(""))     # False – testo vuoto è falso
print(bool([]))     # False – lista vuota è falsa
print(bool(None))   # False

print(bool(1))      # True
print(bool(-5))     # True – qualsiasi numero diverso da zero è vero
print(bool("ciao")) # True – qualsiasi testo non vuoto è vero
```

Cosa succede se la conversione non è possibile?

Se provi a convertire qualcosa che non ha senso, Python ti dà un `ValueError` :

```
int("ciao")      # ValueError – "ciao" non è un numero!
float("abc")     # ValueError
```

Per gestire questo caso senza far crashare il programma, puoi usare un blocco `try/except` (che vedremo in dettaglio nel capitolo sugli errori):

```
valore_inserito = "pippo"
try:
    numero = int(valore_inserito)
    print("Numero valido:", numero)
except ValueError:
    print("Quello che hai scritto non è un numero!")
```

Variabili in Python

Come conservare e usare informazioni nel tuo programma Python.

Cos'è una variabile?

Immagina di avere una serie di scatole etichettate. In ogni scatola puoi mettere un oggetto, e sull'etichetta scrivi il nome per ritrovarla facilmente.

Una **variabile** funziona esattamente così:

- L'etichetta è il **nome** della variabile
- Il contenuto della scatola è il suo **valore**
- La scatola in sé è lo spazio in **memoria** che il computer riserva per conservare quel dato

Creare una variabile

In Python creare una variabile è semplicissimo: basta scegliere un nome e assegnarle un valore con il simbolo `=`.

```
nome = "Alice"      # una scatola chiamata "nome" con dentro il testo "Alice"
eta = 16            # una scatola chiamata "eta" con dentro il numero 16
altezza = 1.65      # una scatola chiamata "altezza" con dentro il numero 1.65
```

Non devi dire a Python che tipo di dato stai mettendo dentro: lo capisce da solo. Non appena scrivi questa riga, la variabile esiste ed è pronta all'uso.

Usare una variabile

Una volta creata, puoi usare il nome della variabile ogni volta che ti serve il suo valore:

```
nome = "Alice"
print(nome)          # Alice
print("Ciao, " + nome + "!") # Ciao, Alice!
```

Cambiare il valore

Puoi cambiare il contenuto di una variabile in qualsiasi momento. È come svuotare la scatola e metterci qualcosa di nuovo:

```
x = 10
print(x)    # 10
x = 25
print(x)    # 25
```

Puoi anche usare il vecchio valore per calcolarne uno nuovo:

```
punteggio = 100
punteggio = punteggio + 50 # oppure: punteggio += 50
print(punteggio)          # 150
```

Assegnare più variabili in una volta

Python ti permette di creare più variabili in una sola riga:

```
a, b, c = 1, 2, 3
print(a)  # 1
print(b)  # 2
print(c)  # 3
```

Oppure puoi assegnare lo stesso valore a più variabili contemporaneamente:

```
x = y = z = 0
print(x, y, z)  # 0 0 0
```

Regole per scegliere i nomi

Non puoi chiamare una variabile in qualsiasi modo. Le regole sono:

- Il nome deve iniziare con una lettera (`a-z` , `A-Z`) oppure con il trattino basso `_`
- Può contenere lettere, numeri e trattini bassi
- **Non può contenere spazi** (usa il trattino basso al loro posto)
- **Non può iniziare con un numero**

- **Non può essere una parola riservata** di Python (come `if`, `for`, `print`, ecc.)

```
# Nomi validi ✓
nome = "Alice"
eta_utente = 16
_privato = 42
valore2 = 3.14

# Nomi non validi ✗
# 2variabile = 10    - inizia con un numero
# mio nome = "Bob"   - contiene uno spazio
# if = "A"           - è una parola riservata di Python
```

Lo stile: snake_case

Per convenzione, i nomi delle variabili in Python si scrivono in **snake_case**: tutto minuscolo, con il trattino basso `_` tra le parole, come una serpe (snake) strisciante.

```
# Stile consigliato ✓
nome_utente = "Alice"
eta_minima = 18
numero_di_studenti = 30

# Stile sconsigliato (funziona, ma non è la norma)
NomeUtente = "Alice"    # CamelCase, si usa per le classi
ETAMINIMA = 18           # tutto maiuscolo, si usa per le costanti
```

Scoprire il tipo di una variabile

Puoi chiedere a Python che tipo di dato contiene una variabile con la funzione `type()` :

```
x = 42
print(type(x))      # <class 'int'> - numero intero

y = "ciao"
print(type(y))      # <class 'str'> - stringa di testo

z = 3.14
print(type(z))      # <class 'float'> - numero decimale
```

Questo torna utile quando non sei sicuro di cosa contiene una variabile.